SALES ANALYTICS

A beginner-friendly map of your Flask project

This guide gathers the explanations from our project walkthrough into one reference: how the app starts, how an upload is cleaned and analyzed, how dashboard charts reach the browser, and how downloads are sent.

> **The most useful way to read the project**
>
> Follow the data values as they move: uploaded file -> DataFrame -> cleaned DataFrame -> analysis dictionary -> chart JSON -> HTML/JavaScript -> browser.

Project landmarks

Area	Main job
app.py	Starts Flask, receives browser requests, coordinates cleaning/analysis, renders templates, and handles downloads.
src/cleaning.py	Takes a DataFrame, applies the project's cleaning steps, and returns a cleaned DataFrame.
src/analysis.py	Takes a DataFrame and returns sales summaries in a Python dictionary.
templates/	HTML pages. Jinja inserts values supplied by Flask.
static/	Browser assets: page styling, the browser Plotly library, and the upload drag/drop script.
data/raw/ and data/cleaned/	Original uploaded files and the reusable cleaned CSV/Excel output.

This is an explanation of the project code that was inspected in this conversation. It does not change your project files. Where the code has a surprising actual behavior, the guide calls that out explicitly.

Quick contents

Section	What you will learn
1. Start and home page	Imports, Flask setup, configuration, and GET /.
2. Upload	How the form posts a file, how Flask saves it, and how Pandas reads it.
3. Cleaning	What clean_sales_data() changes and what it returns.
4. Analysis	How analyze_sales_data() calculates the results dictionary.
5. Upload result/session	Cleaning report, saved cleaned CSV, redirect, and actual/unreachable sections.
6. Dashboard	Filter values, filtered DataFrame, repeated analysis, and no-data path.
7. HTML, JavaScript, Plotly	How chart JSON becomes visible browser charts.
8. Downloads and drag/drop	What both download routes send and how drag/drop fits the form.

A small vocabulary

Word	Meaning in this project
Route	A URL path handled by a Flask function, such as / or /dashboard.
DataFrame	A Pandas table with rows and columns. Examples: df and cleaned_df.
Series	One labeled column-like result, such as sales totals grouped by product.
Dictionary	Key/value data. The analysis result maps names like total_sales to values.
Template	An HTML file in templates/ that Flask fills with supplied values.
Session	Flask's signed browser session storage used here to carry the cleaning report across a redirect.
JSON	Text encoding for structured data that JavaScript can read.

> **Keep names separate**
>
> The variable name df is reused in different functions. Its name is the same, but it is a local variable for that function call. Follow the value passed in and returned, not just the spelling of the variable.

1. What starts the application

The project is started by running app.py. In its main guard, the app runs Flask on the configured host and port; the port comes from the PORT environment variable when present, otherwise 5000. The README's local start command is python app.py.

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 5000)), debug=False)
```

The beginning of app.py

```
from flask import Flask, render_template, request, send_file, redirect, url_for, session
import pandas as pd
import os
import plotly
import plotly.express as px
import json
from src.cleaning import clean_sales_data
from src.analysis import analyze_sales_data

app = Flask(__name__)
app.secret_key = "sales-analytics-secret-key"
UPLOAD_FOLDER = "data/raw"
CLEANED_FOLDER = "data/cleaned"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(CLEANED_FOLDER, exist_ok=True)
```

Import / setting	What it supports later
Flask	Creates the web app object and registers routes.
request	Reads uploaded files and dashboard query/filter values.
render_template	Returns index.html, result.html, or dashboard.html with values.
redirect and url_for	Sends the browser to another route by route name; used after upload and when a result report is missing.
session	Carries cleaning_report between the upload request and the new /result request.
send_file	Returns cleaned CSV or generated Excel as a browser download.
Pandas (pd)	Reads CSV/Excel files, manipulates DataFrames, and writes CSV/Excel.
plotly and px	Build Plotly figure data and encode Plotly-specific values for JSON.
json	Turns a Plotly figure dictionary into JSON text for the template.
clean_sales_data / analyze_sales_data	Connect app.py to the two src modules.
secret_key	Lets Flask sign its session cookie; app.py stores cleaning_report in session.

The two folders are created when app.py starts if they do not already exist. The configured paths are relative to the process's current working directory.

The home route

```
@app.route("/")
def home():
    return render_template("index.html")
```

The browser requests URL /. Flask calls home(), and render_template looks for templates/index.html. The URL / is not the filesystem path to that HTML file: the route maps the URL to the template response.

2. Upload: browser to raw file

The upload page's form sends a multipart POST to /upload. The input's name is file, so the backend reads that exact field name from request.files.

```
<form action="/upload" method="POST" enctype="multipart/form-data">
  <input type="file" name="file" ...>
</form>

@app.route("/upload", methods=["POST"])
def upload_file():
    file = request.files.get("file")
```

POST is used because the browser is submitting file data to the server. The route checks that a file object exists and that its filename is not empty; it returns 'No file selected' if not. It then checks the lowercased extension and accepts .csv or .xlsx, otherwise returning an unsupported-file message. The code does not call secure_filename().

```
file_path = os.path.join(UPLOAD_FOLDER, file.filename)
file.save(file_path)
```

file_path is the server-side location formed from data/raw plus the uploaded filename. file.save() writes the original upload there before Pandas reads it. The original is not the same object as the DataFrame later created from it.

```
if file.filename.lower().endswith(".csv"):
    df = pd.read_csv(file_path)
else:
    df = pd.read_excel(file_path)
```

The extension is checked for reading because CSV and Excel need different Pandas reader functions. Both produce a DataFrame table assigned to app.py's local df. The raw file remains stored in data/raw.

Names to keep distinct

Name	What it is
original upload	The file object received from the browser.
file_path	The path of the saved original, such as data/raw/Sales-data.csv.
df	The DataFrame Pandas read from file_path.
cleaned_df	The cleaned DataFrame returned from clean_sales_data(df).

```
Browser form (field name=file)
    -> POST /upload
    -> request.files.get("file")
    -> file.save(data/raw/<filename>)
    -> file_path
    -> pd.read_csv(...) or pd.read_excel(...)
    -> df
```

> **What is measured before cleaning?**
>
> app.py records before_rows, before_columns, before_duplicates, and before_missing from df. Later it compares row/column/missing counts after cleaning to build the report shown on the result page.

3. cleaning.py: df becomes cleaned_df

app.py imports clean_sales_data from src.cleaning and calls clean_sales_data(df). Inside that function, its parameter is also named df. This is the DataFrame passed by app.py, but the name is local to the cleaning function.

```
def clean_sales_data(df):
    df = df.copy()
    df.columns = df.columns.str.strip().str.replace(" ", "_").str.lower()
    df = df.drop_duplicates()
```

The function begins with a copy, so cleaning operates on a separate DataFrame. It normalizes column labels by trimming, replacing spaces with underscores, and lowercasing; then removes exact duplicate rows. These header changes make later names such as order_date and sales consistent with the code's column checks.

```
text_columns = df.select_dtypes(include="object").columns
for column in text_columns:
    df[column] = df[column].str.strip()
```

It trims leading/trailing whitespace from object/text columns. For a configured list of fields (category, subcategory, brand, city, state, country, region, customer_type, order_status, payment_method, sales_channel), it applies title case when the column exists. Product is not in that title-case list.

```
date_columns = ["order_date", "shipping_date", "delivery_date"]
for column in date_columns:
    if column in df.columns:
        df[column] = pd.to_datetime(df[column], errors="coerce", dayfirst=True)

numeric_columns = ["quantity", "price", "sales", "discount", "tax", "profit", "shipping_cost"]
for column in numeric_columns:
    if column in df.columns:
        df[column] = pd.to_numeric(df[column], errors="coerce")
```

Date fields that exist become Pandas dates; invalid dates are coerced to missing values, and dayfirst=True interprets dates with day before month. Listed numeric fields that exist become numbers; invalid text becomes missing rather than raising a conversion error.

```
if "quantity" in df.columns:
    df.loc[df["quantity"] <= 0, "quantity"] = pd.NA
```

Nonpositive quantity is changed to missing. Separately, when sales, quantity, and price columns all exist, the function runs the following assignments. They are shown indented because they come from inside those column checks.

```
    missing_sales = df["sales"].isna()
    df.loc[missing_sales, "sales"] = df.loc[missing_sales, "quantity"] * df.loc[missing_sales, "price"]

    valid_values = df["quantity"].notna() & df["price"].notna()
    invalid_sales = valid_values & (df["sales"] <= 0)
    df.loc[invalid_sales, "sales"] = df.loc[invalid_sales, "quantity"] * df.loc[invalid_sales, "price"]
```

The first assignment fills missing sales with quantity times price. The second recalculates sales that are zero or negative when quantity and price are present. The function then returns the DataFrame; it does not save a file or create a cleaning report.

> **Back in app.py**
>
> The return value is assigned as cleaned_df = clean_sales_data(df). app.py keeps df as the original in-memory read table and uses cleaned_df for subsequent after-measurements, analysis, and cleaned-file output.

The cleaning function does not generally remove every row with missing data. Its actual operations are the specific conversions and corrections above.

4. analysis.py: cleaned table to results dictionary

app.py calls analyze_sales_data(cleaned_df), both in the upload flow and later in the dashboard flow. The function's parameter named df refers to the DataFrame passed in for that call. It is not automatically the same object as the local df in another function. analysis.py imports pandas as pd, though its active function does not use pd directly.

```
def analyze_sales_data(df):
    results = {}
    results["total_sales"] = df["sales"].sum()
    results["average_sales"] = df["sales"].mean()
    results["total_quantity"] = df["quantity"].sum()
```

It creates a results dictionary and stores total sales, average sales, and total quantity. These are calculated from the sales and quantity columns of the DataFrame it received.

```
product_sales = df.groupby("product")["sales"].sum().sort_values(ascending=False)
results["product_sales"] = product_sales.to_dict()
results["best_product"] = product_sales.idxmax()

city_sales = df.groupby("city")["sales"].sum().sort_values(ascending=False)
results["city_sales"] = city_sales.to_dict()
results["top_city"] = city_sales.idxmax()
```

For each product and each city, groupby groups matching labels, sales.sum() totals each group, and sort_values puts largest totals first. Each grouped Series becomes a dictionary with to_dict(). idxmax() returns the label associated with the largest total.

```
category_sales = df.groupby("category")["sales"].sum().sort_values(ascending=False)
results["category_sales"] = category_sales.to_dict()

df = df.copy()
df["month"] = df["order_date"].dt.to_period("M").astype(str)
monthly_sales = df.groupby("month")["sales"].sum().sort_index()
results["monthly_sales"] = monthly_sales.to_dict()
return results
```

Category sales are grouped and summed too. For monthly sales, the function copies the DataFrame, derives a YYYY-MM month label from order_date, groups sales by month, sums them, sorts month labels, and stores a dictionary. The copy keeps the temporary month column out of the caller's DataFrame.

Returned key	Meaning
total_sales	Sum of sales.
average_sales	Mean of sales.
total_quantity	Sum of quantity.
product_sales	Product label -> summed sales.
best_product	Product label with the maximum summed sales.
city_sales	City label -> summed sales.
top_city	City label with the maximum summed sales.
category_sales	Category label -> summed sales.
monthly_sales	YYYY-MM label -> summed sales.

```
app.py cleaned_df
    -> analyze_sales_data(cleaned_df)
    -> analysis.py calculates summaries
    -> return results dictionary
    -> app.py variable analysis
```

5. Upload after cleaning: report, files, redirect

After cleaned_df is created, app.py calculates after_rows, after_columns, after_missing, rows_removed, and after_duplicates. It also calls analyze_sales_data(cleaned_df) and builds chart data before it writes the cleaned CSV.

In the upload route, the analysis/chart work is performed, but the chart JSON is not actually passed to the user-facing page from this request. The successful path saves the cleaned file, stores the report, then redirects. A redirect ends this function's response before its later template-return code.

```
cleaned_path = os.path.join(CLEANED_FOLDER, "cleaned_sales_data.csv")
cleaned_df.to_csv(cleaned_path, index=False)
```

The saved output is data/cleaned/cleaned_sales_data.csv. to_csv writes the cleaned DataFrame as a CSV; index=False prevents a DataFrame row-number index from being added as an extra column. The original file in data/raw is not overwritten by this save.

```
cleaning_report = {
    "before_rows": before_rows,
    "after_rows": after_rows,
    "before_columns": before_columns,
    "after_columns": after_columns,
    "duplicates": int(before_duplicates),
    "before_missing": before_missing,
    "after_missing": after_missing,
    "rows_removed": rows_removed
}
session["cleaning_report"] = cleaning_report
return redirect(url_for("result"))
```

Those are the report's actual keys. The duplicates value is the before-cleaning duplicate count stored as an integer. The route does not create keys named missing_report, total_fixed, or total_new_missing. The session carries this report to the next request. url_for('result') builds the URL for the result route, and redirect tells the browser to request it.

> **Actual control-flow detail**
>
> The /upload function has a return redirect(...) before a later render_template(...) block. After a successful upload, Python returns at the redirect, so the later code that calculates missing_values, data_types, preview rows/table, and passes chart variables to result.html is unreachable on that path. The active /result route separately renders result.html.

The next request is new

The upload POST returns a redirect response. The browser then sends a separate GET request to /result. Python local variables such as cleaned_df and chart JSON do not carry over automatically. The report is available because app.py explicitly placed it in session.

6. /result: retrieve and show the report

```
@app.route("/result")
def result():
    report = session.get("cleaning_report")
    if not report:
        return redirect(url_for("home"))
    return render_template("result.html", report=report)
```

When the browser follows the redirect, Flask runs result(). It reads cleaning_report from the session. If there is no report (or it is empty), it sends the browser back to home. Otherwise render_template loads templates/result.html and makes the Python dictionary available as the Jinja variable report.

```
POST /upload
  -> original saved, cleaned, analyzed, CSV saved
  -> cleaning_report stored in session
  -> redirect response to /result
  -> browser makes a new GET /result
  -> result() reads session
  -> render_template("result.html", report=report)
  -> browser displays the result page
```

The result page uses report values for the cleaning summary and provides links to the dashboard and download routes. It does not receive the upload-route chart JSON variables. The charts are built and connected during the dashboard route instead. One static bullet list in result.html says records with essential missing values were removed, but the active clean_sales_data() code does not drop rows that way; that bullet is template text, not an operation performed by the function.

Template value example	What Flask gives the template
report.before_rows	The before-cleaning row count.
report.after_rows	The after-cleaning row count.
report.rows_removed	The before minus after row count calculated in app.py.
report.before_missing / after_missing	The corresponding missing-value counts.

> **A useful distinction**
>
> The result route displays the cleaning report, while /dashboard reads the cleaned CSV and computes fresh analysis for whichever filters the user selected.

7. /dashboard: load, filter, analyze

A link to /dashboard causes a GET request. If the cleaned CSV is missing, the route returns 'Please upload a dataset first.' Otherwise it reads the saved cleaned CSV into a DataFrame. It also reads the CSV again for the full lists of filter choices: nonmissing city/category/product values are made unique and sorted. The order_date column is parsed as dates for monthly analysis.

```
city = request.args.get("city", "")
category = request.args.get("category", "")
product = request.args.get("product", "")
```

request.args reads query-string values sent by the GET filter form. The form's select fields are named city, category, and product. Empty string means the user chose the All option, so the corresponding if check skips filtering.

```
if city:
    df = df[df["city"] == city]
if category:
    df = df[df["category"] == category]
if product:
    df = df[df["product"] == product]
```

The route applies active choices one after another. Each step replaces df with only rows matching that exact value. If multiple filters are chosen, the final table must satisfy all of them. For matching rows, it also prepares a preview from the first 100 rows and passes the column names and row values to dashboard.html.

```
if df.empty:
    # render dashboard with zero summary, empty preview, no_data=True
    # chart variables are passed as None
else:
    analysis = analyze_sales_data(df)
    # create chart DataFrames, figures, and JSON
    return render_template("dashboard.html", ...)
```

If there are no matching rows, the route takes the early no-data branch: it does not call analyze_sales_data() and does not build figures. It renders dashboard.html with a no-data flag, empty preview, and chart values None. In the nonempty branch, analyze_sales_data(df) is called again on the currently filtered DataFrame. That is why the displayed totals and charts reflect the selected filters.

No-data page detail

The dashboard template's Plotly JavaScript is placed after the no-data conditional, so it still runs when chart values are None. Jinja inserts the text None, which is not the JavaScript null value; the no-data message is rendered, but the chart script can report an error when it tries to use these values as chart objects.

Four different data states

Original uploaded file = bytes saved in data/raw. Upload df = Pandas read of that original. cleaned_df = cleaning function's return value. Dashboard df = fresh read of cleaned CSV, narrowed by query filters before analysis.

8. Python chart data to visible browser charts

For a nonempty dashboard, app.py takes values in the analysis dictionary and turns each mapping into a chart DataFrame. It creates three Plotly figures: product bar chart, category pie chart, and monthly line chart.

```
product_df = pd.DataFrame(list(analysis["product_sales"].items()), columns=["Product", "Sales"])
product_chart = px.bar(product_df, x="Product", y="Sales", title="Product-wise Sales")
product_chart_json = json.dumps(product_chart.to_dict(), cls=plotly.utils.PlotlyJSONEncoder)
```

Figure	Data supplied
px.bar	Product names and their summed sales from analysis['product_sales'].
px.pie	Category names and summed sales from analysis['category_sales'].
px.line	Month labels and summed sales from analysis['monthly_sales']; markers=True.

px creates a Plotly Figure object on the server; it does not paint pixels in a browser. to_dict() turns its figure specification into ordinary Python dictionary/list data. json.dumps() serializes that structure to JSON text. PlotlyJSONEncoder handles values Plotly may place in the figure structure that the standard JSON encoder does not handle by itself.

```
return render_template("dashboard.html",
    product_chart=product_chart_json,
    category_chart=category_chart_json,
    monthly_chart=monthly_chart_json,
    ...)
```

Flask supplies these values to the Jinja template. dashboard.html loads the local Plotly library and has chart containers with IDs product-chart, category-chart, and monthly-chart. Its inline JavaScript inserts the JSON and calls Plotly.newPlot() for each container.

```
<script src="{{ url_for('static', filename='js/plotly.min.js') }}"></script>
<script>
  const productData = {{ product_chart | safe }};
  Plotly.newPlot("product-chart", productData.data, productData.layout, {responsive: true});
</script>
```

Jinja replaces the template expression with the value from Flask before the browser receives the HTML. The safe filter keeps the JSON text from being HTML-escaped. The browser JavaScript parses the JSON object, and Plotly.newPlot uses its data and layout to draw the chart.

```
analysis dictionary
 -> chart DataFrame
 -> Plotly Figure
 -> Figure dictionary
 -> JSON text
 -> render_template variables
 -> Jinja-generated HTML/JavaScript
 -> Plotly.newPlot()
 -> visible chart
```

> **Browser scripts to distinguish**
>
> static/js/plotly.min.js is loaded by dashboard.html and provides Plotly in the browser. static/js/script.js is now loaded by index.html and handles drag/drop. The upload filename display and dashboard table search are inline in their templates.

9. Actual chart connection and downloads

There are two analysis/chart moments. The upload route calls analysis and constructs chart JSON, but then it stores the cleaning report and redirects to /result. It does not carry those local JSON variables through the redirect, and result.html does not use them. On /dashboard, the route reads the cleaned CSV again, applies filters, analyzes the filtered rows, builds chart JSON again, and passes it to dashboard.html. That dashboard path is the working chart connection.

CSV download route

```
@app.route("/download")
def download_file():
    cleaned_path = os.path.join(CLEANED_FOLDER, "cleaned_sales_data.csv")
    if not os.path.exists(cleaned_path):
        return "Cleaned dataset not found. Please upload a dataset first."
    return send_file(cleaned_path, as_attachment=True, download_name="cleaned_sales_data.csv")
```

The Download CSV links on result.html and dashboard.html request /download. send_file returns the cleaned CSV as an attachment. The route uses data/cleaned/cleaned_sales_data.csv, not the original upload in data/raw. The browser receives the file and downloads it or offers its normal save behavior.

Excel download route

```
@app.route("/download-excel")
def download_excel():
    cleaned_path = os.path.join(CLEANED_FOLDER, "cleaned_sales_data.csv")
    if not os.path.exists(cleaned_path):
        return "Cleaned dataset not found. Please upload a dataset first."
    df = pd.read_csv(cleaned_path)
    excel_path = os.path.join(CLEANED_FOLDER, "cleaned_sales_data.xlsx")
    df.to_excel(excel_path, index=False)
    return send_file(excel_path, as_attachment=True, download_name="cleaned_sales_data.xlsx")
```

The Excel link requests /download-excel. The route reads the cleaned CSV, writes its contents to cleaned_sales_data.xlsx with no index column, then sends that Excel file. It generates or overwrites the XLSX on each request; it does not send the original Excel upload, even if one was uploaded.

Browser link	Server file used	What the user downloads
/download	data/cleaned/cleaned_sales_data.csv	Cleaned CSV attachment.
/download-excel	Reads cleaned CSV, writes data/cleaned/cleaned_sales_data.xlsx	Generated cleaned Excel attachment.

Drag and drop note

The current index.html has a drop-hint paragraph inside the upload form, but the paragraph itself is empty. The visible words are supplied by style.css through .drop-hint::before, which inserts the text 'Or drag and drop a CSV or XLSX file here'. index.html loads static/js/script.js; that file listens for drag events, validates .csv/.xlsx, places the dropped file into the existing input using DataTransfer, and calls showFileName(). The same form field named file is then POSTed to /upload, so the backend flow stays the same. The hint does not depend on JavaScript; if it is still absent, inspect whether the page receives the CSS and refresh the browser cache.

10. The complete project flow

This diagram follows the actual paths through the project. The upload and dashboard are separate browser requests. The result report travels through the session; chart JSON is generated again on the dashboard request.

```
START
python app.py
  -> Flask app created and routes registered

HOME PAGE
Browser GET /
  -> home()
  -> render templates/index.html

UPLOAD AND CLEAN
index.html form POST /upload (multipart field: file)
  -> request.files
  -> save original under data/raw/<filename>
  -> Pandas reads CSV/XLSX into upload df
  -> measure before-cleaning counts
  -> clean_sales_data(df)
  -> cleaned_df
  -> measure after-cleaning counts
  -> analyze_sales_data(cleaned_df) and upload-route chart JSON is built
  -> save cleaned_df to data/cleaned/cleaned_sales_data.csv
  -> create cleaning_report and put it in session
  -> redirect browser to GET /result

RESULT PAGE
/result reads cleaning_report from session
  -> render templates/result.html with report
  -> user can open dashboard or download files

DASHBOARD AND CHARTS
Browser GET /dashboard?city=...&category=...&product=...
  -> read cleaned CSV (and full lists for filter choices)
  -> read query filters and filter DataFrame
  -> if rows remain: analyze_sales_data(filtered df)
  -> chart DataFrames -> Plotly Figures -> JSON strings
  -> render templates/dashboard.html with summary, filters, preview, chart JSON
  -> Jinja inserts JSON into JavaScript
  -> browser loads static/js/plotly.min.js
  -> Plotly.newPlot() draws charts

DOWNLOADS
/download -> send cleaned_sales_data.csv
/download-excel -> read cleaned CSV -> generate cleaned_sales_data.xlsx -> send it
```

Files and their actual roles

Item	Actual role in the inspected project
app.py	Entry point and request coordinator.
cleaning.py	Used by /upload; returns a DataFrame.
analysis.py	Used after cleaning and in the dashboard; returns a dictionary.
index.html	Home upload form.
result.html	Displays report supplied by /result and links onward.
dashboard.html	Receives filtered metrics/preview/chart JSON and runs Plotly rendering JavaScript.
plotly.min.js	Browser Plotly library loaded by dashboard template.
script.js	Loaded by index.html; handles drag/drop for the upload input.
data/raw files	Original uploaded files saved by /upload.

11. Last-page review

Use this page as a quick route-and-value reference after you finish the full walkthrough.

Route	Browser request	What the route does
/	GET	Shows templates/index.html and the upload form.
/upload	POST	Reads file field, saves original, reads/cleans/analyses data, saves cleaned CSV, stores report, redirects.
/result	GET	Reads cleaning_report from session and renders result.html.
/dashboard	GET	Reads cleaned CSV, applies query filters, analyzes matches, sends chart JSON to dashboard.html.
/download	GET	Sends the cleaned CSV as an attachment.
/download-excel	GET	Reads cleaned CSV, creates/overwrites cleaned XLSX, and sends it.

Step	Input type	Output type
clean_sales_data(df)	DataFrame	DataFrame (cleaned copy)
analyze_sales_data(df)	DataFrame	Dictionary of summaries and grouped totals
Chart preparation	Analysis dictionary	Plotly Figure, then JSON text
render_template dashboard	Python values / JSON	HTML and JavaScript sent to browser

Supporting-file status

script.js is now used by index.html for drag/drop. Plotly is loaded by dashboard.html. The existing cleaned_sales_data.xlsx is not read by a download route; /download-excel regenerates it from the CSV. README.md and requirements.txt support setup and dependency installation, rather than handling browser requests.

Five ideas to remember

1) Flask routes connect URLs to Python functions. 2) cleaning.py returns a DataFrame; analysis.py returns a dictionary. 3) the redirect creates a new request, so the report is explicitly kept in session. 4) dashboard filters cause fresh analysis of the filtered rows. 5) browser charts need the JSON, the Plotly library, and the template JavaScript call.

Quick self-check

- Why can the dashboard show a different sales total after a city filter?
- Which object carries the cleaning report from POST /upload to GET /result?
- Which route reads the cleaned CSV and generates the Excel download?
- Where is Plotly.newPlot() run: in Python or in the browser?

Answer key: dashboard analysis runs on the filtered DataFrame; the Flask session carries the report; /download-excel reads the cleaned CSV and creates the XLSX; Plotly.newPlot() runs in browser JavaScript.

End of guide